

SBFT Tool Competition 2026 – UAV Testing Track

Erdem Uysal
University of Bern
Bern, Switzerland
ramazan.uysal@unibe.ch

Aren Babikian
University of Toronto
Toronto, Canada
aren.babikian@utoronto.ca

Ali Javadi
University of Bern
Bern, Switzerland
ali.javadi@unibe.ch

Dmytro Humeniuk
Polytechnique Montréal
Montréal, Canada
dmytro.humeniuk@polymtl.ca

Prakash Aryan
University of Bern
Bern, Switzerland
prakash.aryan@unibe.ch

Sajad Mazraehkhatiri
University of Bern
Bern, Switzerland
sajad.mazraehkhatiri@unibe.ch

Sebastiano Panichella
University of Bern
Bern, Switzerland
AI4I - The Italian Institute of Artificial
Intelligence for Industry
Turin, Italy
sebastiano.panichella@unibe.ch

Abstract

Unmanned Aerial Vehicles (UAVs) are complex, safety-critical cyber-physical systems for which simulation-based testing plays a vital role in validating reliable and safe behavior. Despite its widely acknowledged importance, UAV testing remains an underexplored research area with considerable growth potential. The UAV Testing Competition aims to foster greater involvement from the software testing community and to establish cyber-physical systems testing as an important and evolving research field. The latest edition of the competition was jointly organized by SBFT and ICST and introduced wind disturbances into the simulator to present environmental challenges. In this edition, we received and evaluated a single submission for the SBFT track. The submitted tool's test generation capabilities were assessed across multiple case studies and compared against the baseline tool, Surrealist. The evaluation focused on both the diversity of generated test cases and the effectiveness in detecting failures in PX4-Autopilot.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; • **Computer systems organization** → *Robotics*.

Keywords

Tool Competition, Automated Testing, Search-Based Software Engineering, Unmanned Aerial Vehicles

ACM Reference Format:

Erdem Uysal, Ali Javadi, Prakash Aryan, Aren Babikian, Dmytro Humeniuk, Sajad Mazraehkhatiri, and Sebastiano Panichella. 2026. SBFT Tool Competition 2026 – UAV Testing Track. In *19th Search-Based and Fuzz Testing*



This work is licensed under a Creative Commons Attribution 4.0 International License. *SBFT '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2387-2/26/04
<https://doi.org/10.1145/3786155.3795696>

Workshop (SBFT '26), April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3786155.3795696>

1 Introduction

Unmanned Aerial Vehicles (UAVs) are widely adopted, demonstrating efficient and autonomous capabilities across various domains. Open-source hardware and software projects, such as ArduPilot and PX4 [11], have emerged and contributed significantly to advancing the autonomous capabilities of these complex cyber-physical systems. However, testing the safety-critical functionalities of such systems to ensure reliable and safe operation has progressed more slowly [1, 15].

Simulation-based testing is essential for guaranteeing UAV safety [1, 3]. When conducted effectively, it can accurately identify bugs and issues in UAV systems, thereby significantly enhancing the safety and reliability of subsequent field testing [6]. However, accurately replicating physical testing environments and detecting bugs that may arise in real-world scenarios remain significant challenges [6–8].

The UAV Testing Competition provides software testing researchers with a structured platform designed to facilitate simulation-based UAV testing. This platform motivates and enables researchers to apply search-based test generation techniques in a novel and challenging domain: *autonomous vision-based UAV navigation systems*. Building upon the success of earlier testing tool competitions, we invited researchers to participate by developing their own system-level test generation tools for UAVs and competing against each other as well as the state-of-the-art baseline [7].

Building on the strong interest generated by the inaugural UAV Testing Competition in 2024[9] and incorporating feedback and insights from the second edition in 2025 [10], we made several improvements to the infrastructure and introduced environmental challenges to enhance this year's competition. This 2026 edition was jointly organized by the SBFT and ICST communities [14]. One

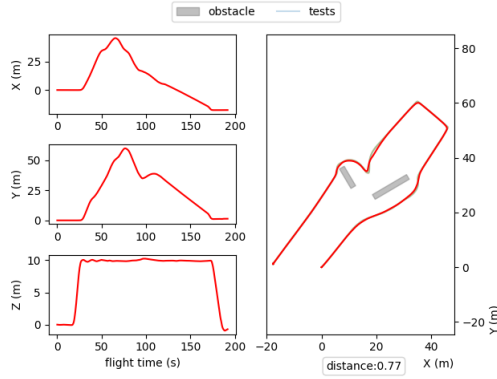


Figure 1: A failing test case

tool was submitted to SBFT 2026 (described in this paper). ICST 2026 submissions are detailed in the corresponding competition report [14].

2 Competition Design

As part of the competition, participants are expected to develop a test generator for the given system under test, PX4 [11], a vision-based autonomous flight system. The submitted tool should be capable of generating diverse and effective test cases to identify underlying system vulnerabilities. To identify vulnerabilities in the system under test, the submitted tool should modify the placement and size of obstacles within the simulated environment. A test will be considered valid if the UAV deviates onto an unsafe trajectory or collides with an obstacle, as illustrated in Figure 1. The generated tests are evaluated by analyzing the minimum distance between the trajectory of the UAV and the obstacles present in the environment.

Participants were given a test infrastructure tool [8] to develop their own test generator tool on top of that:

<https://github.com/skhatiri/UAV-Testing-Competition>

2.1 Benchmarking Platform

PX4 [11], a vision-based UAV autopilot system, is the software under test. This open-source platform has been thoroughly examined in previous research [4, 5, 7]. The UAV test bench Aerialist [8], built on top of PX4, facilitates the testing process.

2.1.1 PX4 Platform. PX4 is an open-source autopilot with a global community, supporting autonomous control and navigation for ground, air, and underwater vehicles using a wide range of sensors.

The PX4-Avoidance module enables safe UAV navigation by avoiding obstacles. PX4 also records detailed flight data and telemetry for in-depth analysis and supports SITL simulations such as Gazebo and JMAVSIM to develop and test advanced UAV features.

2.1.2 Aerialist. Aerialist¹ (unmanned AERIAL vehicle teST bench) is an advanced UAV testing platform designed to streamline and automate the testing workflow. Aerialist offers participants an intuitive and efficient platform that simplifies UAV test execution [8], allowing them to focus primarily on developing their test generation tool.

¹<https://github.com/skhatiri/Aerialist>

Aerialist represents a UAV test case as a structured set of test properties formatted in YAML. This structure includes *drone* properties (such as software configurations and mission plans), *simulation* properties (covering the simulator, environment, and obstacles), and *commands* executed during runtime. Aerialist can also be executed in a containerized environment using Docker, whether locally or in the cloud, with Kubernetes supporting test execution on a large scale.

2.2 Rules and Restrictions

The submitted test generators by the participants should generate challenging test cases and find vulnerabilities in the UAV system for a given case study.

A case study, represented in YAML format, is created by specifying an Aerialist test description, *i.e.*, a mission plan, a predefined UAV configuration, and a simulation environment without obstacles. Test generators can introduce **up to three obstacles** into the simulator. Each obstacle is defined by its dimensions (length, width, height in meters), position within the simulation environment (x , y , z), and rotation angle (r) in degrees.

The autonomous UAV should complete the mission by avoiding the obstacles placed along the trajectory in the environment. The test generator is expected to generate tests that make it hard for the UAV to navigate correctly in the environment and increase the chance of collision or following an unsafe trajectory. The obstacles introduced by the tool may cause the UAV to deviate from its planned trajectory, bringing it dangerously close—within the 1.5m safety threshold [7]—while still completing the mission, or potentially lead to a crash. These failing test cases reveal potential vulnerabilities that warrant further investigation by the UAV developers (as demonstrated in Figure 1).

Participants use search-based techniques to identify complex obstacle configurations. Generated test cases must follow Aerialist’s modeling framework and meet these requirements:

- Any test case will be deemed invalid if it creates an impossible navigation scenario, such as placing an obstacle as significant as a long wall that prevents the mission from being completed. The path planning should remain physically feasible.
- All obstacles must be confined within the designated rectangular area defined in the case study.
- Unlike last year’s edition, only up to three obstacles are allowed in the environment at a time, increasing the challenge of finding failing test cases. They must be placed directly on the ground ($z = 0$), have a height greater than the UAV’s flight altitude ($h > 10m$), and must not overlap.

3 Experiments and Results

3.1 Competing Tools

Intelligent-UAV-Gen [12] is a tool submitted to the competition. Surrealist [7], the state-of-the-art UAV test generation approach, was used as the baseline.

Intelligent-UAV-Gen [12] leverages large language models to produce failure revealing test seeds and explores the search areas close to them by performing mutations.

Surrealist [7] employs an iterative adaptive greedy search approach to generate test cases that maximize a difficulty measure defined based on the drone's minimum distance to the obstacles.

3.2 Evaluation Process

The tools were evaluated in two stages: generating a test suite for each case study within a 100-evaluation budget, then assessing and scoring the results.

3.2.1 Test Generation Phase.

Case Studies. We used four case studies (CS) of increasing difficulty to evaluate the test generation approaches. Adapted from the most challenging cases in the previous edition [9], they vary by number of waypoints (3,4), overlap with obstacle areas, and mission plan shape (rectangular, triangular). Figure 1 shows a sample flight with four waypoints, a rectangular shape, and two obstacles.

Tool Execution. All submitted tools were executed in a hybrid setup, with the tool itself running directly on a server with 16 vCPUs and 32 GB RAM, and the simulation running in a Docker container. During evaluation, Kubernetes was used to orchestrate the simulations and ensure consistency across tools. For each case study, the simulation budget for each tool was set to 100, representing the maximum number of test cases allowed for simulation. To ensure consistency, the test cases were executed in separate, isolated containers with fixed resource allocations (6 vCPUs, 4 GB RAM). For each simulation, a timeout of 500 seconds was allocated, after which the simulation was terminated, and logs were extracted. After each test case execution, the tools are expected to generate a test suite containing the failing tests found during the execution and ordered based on their relevance and criticality. These failing tests were then used for the final evaluation.

3.2.2 Test Suite Evaluation Phase. Due to time and computational constraints, evaluations were limited to 20 generated tests per tool for each case study, selected sequentially from the test generator output. These tests were first checked for compliance with competition rules (e.g., obstacle size and placement). Valid tests were then executed three times to address non-determinism, with each run scored using the minimum distance between the UAV trajectory and obstacles (min_dist) as per the Equation 1 shown below.

$$\text{point}(\text{sim}) = \begin{cases} 5, & \text{if } \text{min_dist}(\text{sim}) < 0.25m \\ 2, & \text{if } 0.25m \leq \text{min_dist}(\text{sim}) < 1m \\ 1, & \text{if } 1m \leq \text{min_dist}(\text{sim}) < 1.5m \\ 0, & \text{if } \text{min_dist}(\text{sim}) \geq 1.5m \end{cases} \quad (1)$$

The test_score for each test case was calculated using its average point (avg_point), as shown in Formula 2. To encourage scenarios with fewer obstacles—a more challenging task—we penalized test cases with more obstacles and longer test execution times (avg_time) in minutes.

$$\text{test_score}(t) = \frac{\text{avg_point}(t) \times 10}{\#obst(t)^2 \times \text{avg_time}(t)} \quad (2)$$

The average score Failure_score of a test suite s was calculated as the sum of all the test scores (excluding duplicates):

$$F_s = \sum_{t \in s} \text{test_score}(t) \quad (3)$$

To quantify the *input diversity* of a generated test suite, we define a geometry-based diversity metric that captures both the spatial coverage of test scenarios and the variability in their geometric extent. Let $\mathcal{G} = \{G_1, G_2, \dots, G_n\}$ be a set of $n \geq 2$ test geometries, where each geometry represents an obstacle that is part of the test scenario, and let $A(G_i)$ denote the area of geometry G_i .

First, we measure the overall spatial coverage of the test by computing the ratio between the area of the union of all geometries and the average area of individual geometries:

$$D_{\text{union}} = \frac{A(\bigcup_{i=1}^n G_i)}{\frac{1}{n} \sum_{i=1}^n A(G_i)}.$$

This term penalizes redundant test scenarios with overlapping obstacles while rewarding tests that collectively cover a larger portion of the test area.

Second, to account for structural variability among test scenarios, we compute the coefficient of variation of the geometry areas:

$$\text{CV} = \frac{\sqrt{\frac{1}{n-1} \sum_{i=1}^n (A(G_i) - \bar{A})^2}}{\bar{A}}, \quad \text{where } \bar{A} = \frac{1}{n} \sum_{i=1}^n A(G_i).$$

Intuitively, we want to assign increased reward to tests that contain obstacles with varying (diverse) areas.

Finally, the overall diversity of the test suite is defined as:

$$D_{\text{in}} = D_{\text{union}} \cdot (1 + \text{CV}).$$

If all obstacles present in all tests of a test suite have the same area, the diversity metric D will equal the initially computed D_{union} . Variations in obstacle area within a test suite lead to an increased coefficient of variation CV and to a higher final diversity metric D_{in} , thus promoting diversity in obstacle areas.

Apart from the test suite *input diversity*, we also compute the test suite *output diversity*, characterized by the difference in the test subject trajectories across the tests for the same mission.

We compute it as follows. We assume having $m > 1$ generated test scenarios.

We re-run the scenario evaluation 3 times for each test j , producing trajectories:

$$\mathcal{T}_j = \{\tau_{j,1}, \tau_{j,2}, \tau_{j,3}\}, \quad j = 1, \dots, m$$

For each test we compute the averaged trajectory $\bar{\tau}_j$

$$\bar{\tau}_j = \text{Avg}(\mathcal{T}_j)$$

As a result, we obtain m averaged trajectories:

$$\bar{\mathcal{T}} = \{\bar{\tau}_1, \bar{\tau}_2, \dots, \bar{\tau}_m\}$$

We then compute the nearest neighbor dynamic time warping (DTW) distance δ_j for each test:

$$\delta_j = \min_{\substack{k=1 \\ k \neq j}}^m \text{DTW}(\bar{\tau}_j, \bar{\tau}_k)$$

We then average the minimal nearest neighbor distance across the tests, resulting in a final *output diversity* metric:

$$D_{out} = \frac{1}{m} \sum_{j=1}^m \delta_j$$

For each case study CS_i , tools were ranked independently as per *failure* (R_F), *input diversity* ($R_{D_{in}}$) and *output diversity* ($R_{D_{out}}$) scores of their generated test suites as reported in Table 1. The individual ranks R_F , $R_{D_{in}}$ and $R_{D_{out}}$ were then aggregated to find the top-performing tool in each category (failure detection and test diversity). The final ranking was determined using Formula 4, which assigns two times more weight to the failure detection performance, as uncovering high-severity failures (i.e., those involving closer proximity to obstacles) remains the competition's primary focus.

$$R = 0.5 \times R_F + 0.25 \times R_{D_{in}} + 0.25 \times R_{D_{out}} \quad (4)$$

3.3 Results and Ranking

Table 1 summarizes our evaluation metrics. For each case study, the total number of produced test cases, the number of failed test cases among the 20 evaluated ones (in parentheses), the failure, input and output diversity scores, as well as the final tool ranking are reported. Individual scores are averaged in the final column. More details, including the case study definitions, evaluated test suites, flight plots, and scoring details, are included in the evaluation artifacts [13].

All evaluated tools could generate valid failing test cases ($\#tests > 0$) for all the 3 case studies. At the same time, due to the non-determinism involved in the performance of the PX4-Autopilot agent and the simulation environment, not all reported tests that were failing during the test generation phase (simulated once) also failed in the evaluation phase (simulated three times). These remarks highlight the importance of considering non-determinism in the test generation approaches.

The results indicate that **Intelligent-UAV-Gen** consistently achieves the highest failure scores across all use cases, while maintaining strong input and output diversity, leading to the best overall average and a first-place ranking. **Surrealist** ranks second, generating a larger number of tests in several use cases and exhibiting competitive diversity scores, but overall lower failure scores. These results suggest that Intelligent-UAV-Gen is more effective at generating failure-revealing tests while preserving diversity.

4 Conclusion and Final Remarks

This year marked the third edition of the UAV Testing Competition, jointly organized at ICST and SBFT 2026. We evaluated and compared one tool at SBFT using Surrealist as a baseline, with the proposed tool emerging as the top performer. This edition introduced wind disturbances in the simulator to increase testing challenges and a new input diversity metric based on obstacle shape similarity. The 2026 competition was limited to box-shaped obstacles. Future editions will introduce new metrics [2] and expand to diverse obstacle types, such as trees and buildings, as well as environmental factors.

Acknowledgments

We thank all the participants for their contribution to the competition. We thank the Horizon 2020 (EU Commission) support for the project InnoGuard, Marie Skłodowska-Curie Actions Doctoral Networks (HORIZON-MSCA-2023-DN), the Hasler Foundation for the project "Safe2Fly" (No. 2025-02-27-311), and the SNSF for the project entitled "SwarmOps: Human-sensing based MLOps for Collaborative Cyber-physical systems" (Project No. 200021_219732).

References

- [1] Christian Birchler, Sajad Khatiri, Pooja Rani, Timo Kehrler, and Sebastiano Panichella. 2025. A Roadmap for Simulation-Based Testing of Autonomous Cyber-Physical Systems: Challenges and Future Direction. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 152:1–152:9. doi:10.1145/3711906
- [2] Christian Birchler, Tanzil Kombarabettu Mohammed, Pooja Rani, Teodora Nechita, Timo Kehrler, and Sebastiano Panichella. 2024. How Does Simulation-Based Testing for Self-Driving Cars Match Human Perception? *Proc. ACM Softw. Eng.* 1, FSE (2024), 929–950. doi:10.1145/3643768
- [3] Christian Birchler, Cyrill Rohrbach, Timo Kehrler, and Sebastiano Panichella. 2024. SemoDat: Simulation-based Sensor Dataset of Self-driving Cars. In *21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15–16, 2024*, Diomidis Spinellis, Alberto Bacchelli, and Eleni Constantinou (Eds.). ACM, 510–514. doi:10.1145/3643991.3644891
- [4] Andrea Di Sorbo, Fiorella Zampetti, Corrado A. Visaggio, Massimiliano Di Penta, and Sebastiano Panichella. 2022. Automated Identification and Qualitative Characterization of Safety Concerns Reported in UAV Software Platforms. *ACM Trans. Softw. Eng. Methodol.* (Sept. 2022). doi:10.1145/3564821
- [5] Sajad Khatiri, Fatemeh Mohammadi Amin, Sebastiano Panichella, and Paolo Tonella. 2025. When uncertainty leads to unsafety: Empirical insights into the role of uncertainty in unmanned aerial vehicle safety. *Empir. Softw. Eng.* 30, 6 (2025), 166. doi:10.1007/S10664-025-10697-Z
- [6] Sajad Khatiri, Francisco Eli Vina Barrientos, Maximilian Wulf, Paolo Tonella, and Sebastiano Panichella. 2025. Bridging Research and Practice in Simulation-based Testing of Industrial Robot Navigation Systems. (2025).
- [7] Sajad Khatiri, Sebastiano Panichella, and Paolo Tonella. 2023. Simulation-based test case generation for unmanned aerial vehicles in the neighborhood of real flights. In *International Conference on Software Testing, Verification and Validation*.
- [8] Sajad Khatiri, Sebastiano Panichella, and Paolo Tonella. 2024. Simulation-based Testing of Unmanned Aerial Vehicles with Aerialist. In *International Conference on Software Engineering (ICSE)*.
- [9] Sajad Khatiri, Prasun Saurabh, Timothy Zimmermann, Charith Munasinghe, Christian Birchler, and Sebastiano Panichella. 2023. "SBFT Tool Competition on Testing Unmanned Aerial Vehicles". <https://github.com/skhatiri/UAV-Testing-Competition>
- [10] Sajad Khatiri, Tahereh Zohdinasab, Prasun Saurabh, Dmytro Humeniuk, and Sebastiano Panichella. 2025. ICST Tool Competition 2025 - UAV Testing Track. In *IEEE/ACM International Conference on Software Testing, Verification and Validation, ICST 2025*.
- [11] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. 2015. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *international conference on robotics and automation*. IEEE, 6235–6240.
- [12] Arham Riaz, Taohong Zhu, and Youcheng Sun. 2026. CAAI-ST: Constraint-Aware AI-Guided Seed Generation and Mutation for CPS-UAV System Testing. In *International Workshop on Search-Based and Fuzz Testing, SBFT@ICSE 2026*.
- [13] Erdem Uysal, Ali Javadi, Prakash Aryan, Aren Babikian, Dmytro Humeniuk, Sajad Khatiri, and Sebastiano Panichella. 2026. UAV Testing Competition 2026: Evaluation Results. <https://zenodo.org/doi/10.5281/zenodo.18492162>. doi:10.5281/ZENODO.18492162
- [14] Ramazan Erdem Uysal, Ali Javadi, Prakash Aryan, Aren Babikian, Dmytro Humeniuk, Sajad Mazraehkhatiri, and Sebastiano Panichella. 2026. ICST Tool Competition 2026 – UAV Testing Track. In *International Conference on Software Testing, Verification and Validation (ICST)*.
- [15] Fiorella Zampetti, Ritu Kapur, Massimiliano Di Penta, and Sebastiano Panichella. 2022. An empirical characterization of software bugs in open-source Cyber-Physical Systems. *J. Syst. Softw.* 192 (2022), 111425. doi:10.1016/J.JSS.2022.111425

Table 1: Ranking of the tools

Tool Name	#Obst.	#tests	CS2			#tests	CS3			#tests	CS4			#tests	CS5			Average			Final Rank
			F_s	D_{in}	D_{out}		F_s	D_{in}	D_{out}		F_s	D_{in}	D_{out}		F_s	D_{in}	D_{out}	F_s	D_{in}	D_{out}	
Intelligent-UAV-Gen	2	66 (20)	19.226	4.251	7.718	86 (19)	11.228	3.568	34.87	16 (4)	3.12	3.463	4.355	77 (19)	25.32	3.28	5.029	14.725	3.641	12.99	1
Surrealist	2	80 (17)	18.251	1.29	1.525	108 (19)	26.276	1.62	14.74	13 (4)	2.44	1.34	3.595	79 (0)	0	1.436	7	11.742	1.425	6.716	2